Universidad Cuenca del Plata - Sede Posadas

Paradigmas y Lenguajes de Programación III

Actividad N°2: (13/10) - Desafío Práctico de 3 Tests

Alumnos: Noguera David y Viarengo Gonzalo

Proyecto: Savage Grill (API REST de gestión gastronómica)

Fecha de realización: 14/10/2026

1. Descripción

Se realizaron las tres pruebas pedidas en la consigna sobre la API del proyecto Savage Grill, usando Postman contra el servidor local (http://localhost:8080/api/v1). Las pruebas se hicieron con el usuario jperez, que tiene rol Mozo, usando el token que devuelve el login.

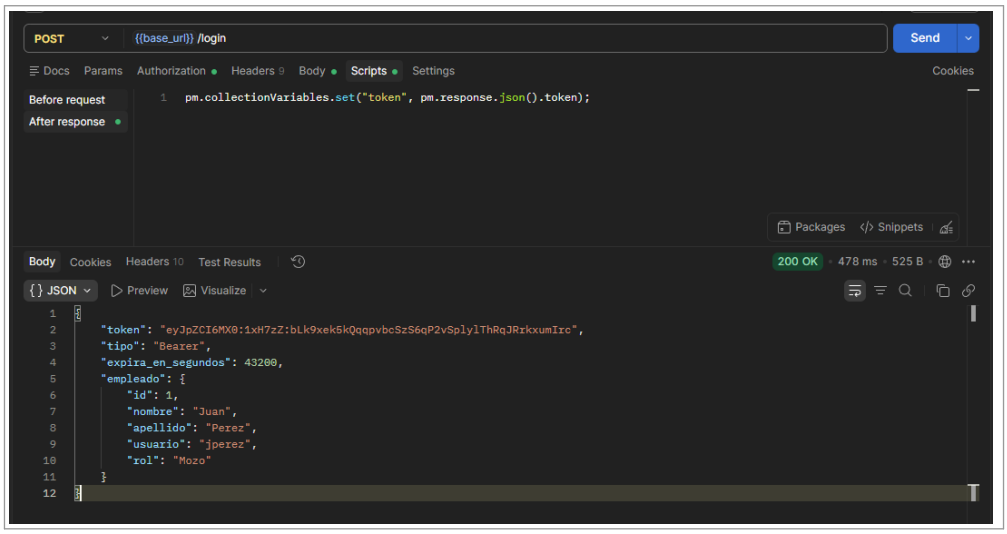
Orden en que se ejecutaron: login, Prueba 2, Prueba 1 y Prueba 3. La Prueba 1 usa la sesión de mesa que abre la Prueba 2.

2. Resumen de resultados

Prueba	Resultado esperado	Resultado obtenido
1. Regla de negocio (subtotal del pedido)	Status 201. El servidor calcula sub_total = precio_unitario x cantidad e ignora el precio (1) y el subtotal (2) que manda el cliente.	Status 201. Las 3 verificaciones pasaron (PASSED).
2. Integración 201 (abrir mesa)	Status 201, Content-Type JSON, schema JSON correcto y mesa 2 ocupada en la base de datos.	Primera ejecución: 201, pero 2 de 3 verificaciones (falló Content-Type por un error del script de prueba). Segunda ejecución, con el script corregido: 201, 3 de 3 PASSED y mesa 2 con Estado = 1.
3. Borde 400 (3 requests: A, B y C)	Status 400 Bad Request con un JSON que explica el error, sin error 500.	Los 3 requests devolvieron 400 con un mensaje claro. En cada uno pasaron las 3 verificaciones.

3. Preparación: login

Esperado: que POST /login con un usuario de prueba devuelva 200 y un token. Obtenido: 200 OK con el token y el rol Mozo. Un script en Post-response guarda el token en una variable para usarlo en los demás requests.

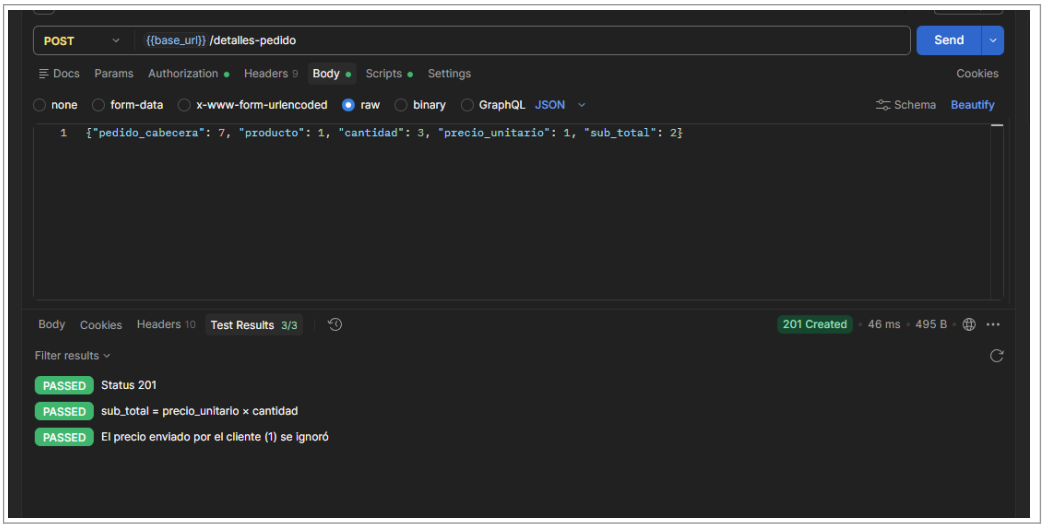


Captura 1. Login: respuesta 200 OK y script que guarda el token.

4. Prueba 1: regla de negocio (cálculo del subtotal)

Un test unitario puro no se ejecuta por HTTP, por eso en Postman se verificó la misma regla de negocio a través de la API: el servidor, y no el cliente, calcula el precio y el subtotal de cada renglón del pedido. En el request se enviaron a propósito un precio (1) y un subtotal (2) falsos, sobre la sesión abierta con id 7.

Esperado: status 201, sub_total igual a precio_unitario por 3, y que el precio enviado por el cliente se ignore. Obtenido: status 201 y las 3 verificaciones en PASSED.



Captura 2. POST /detalles-pedido con precio y subtotal falsos: 201 Created y 3 de 3 verificaciones aprobadas.

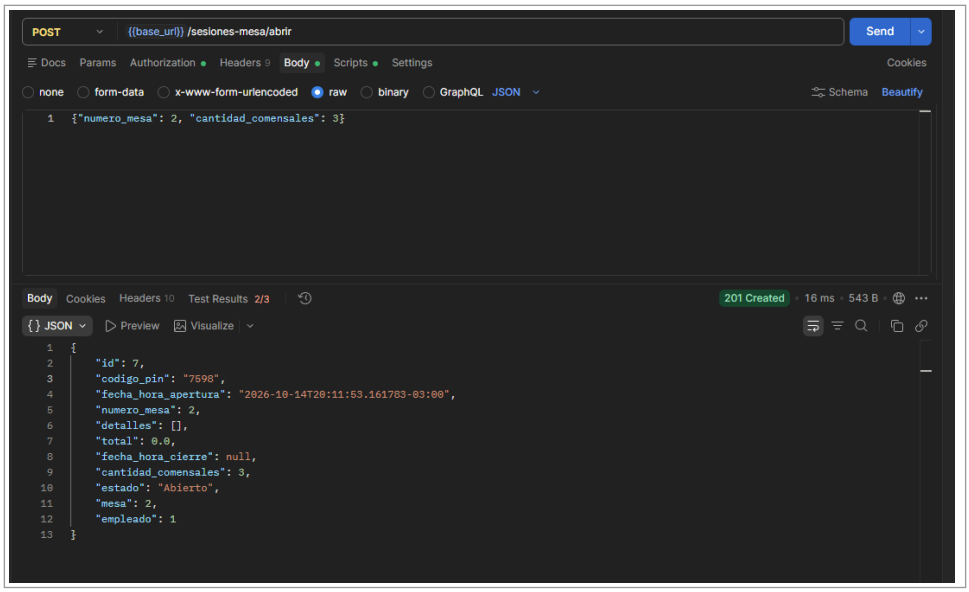
5. Prueba 2: integración con status 201 (abrir mesa)

Se envió un POST a /sesiones-mesa/abrir con un payload válido: {"numero_mesa": 2, "cantidad_comensales": 3}. La prueba recorre el flujo completo: token, permisos por rol, validación de datos y base de datos.

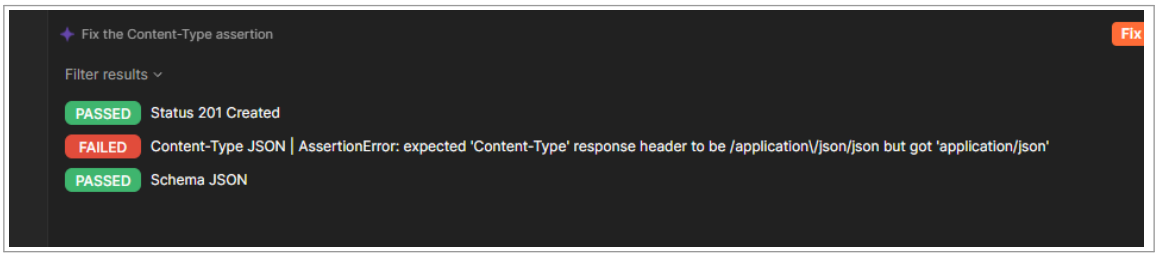
Esperado: status 201, Content-Type application/json, schema JSON con todos los campos y sus tipos, estado "Abierto", PIN de 4 dígitos y mesa ocupada.

Primera ejecución

La API respondió 201 correctamente, pero la verificación de Content-Type falló. El error estaba en el script del test, que comparaba el encabezado con una expresión regular y Postman espera el texto exacto. La API sí devolvió application/json. Se corrigió el script comparando con "include".



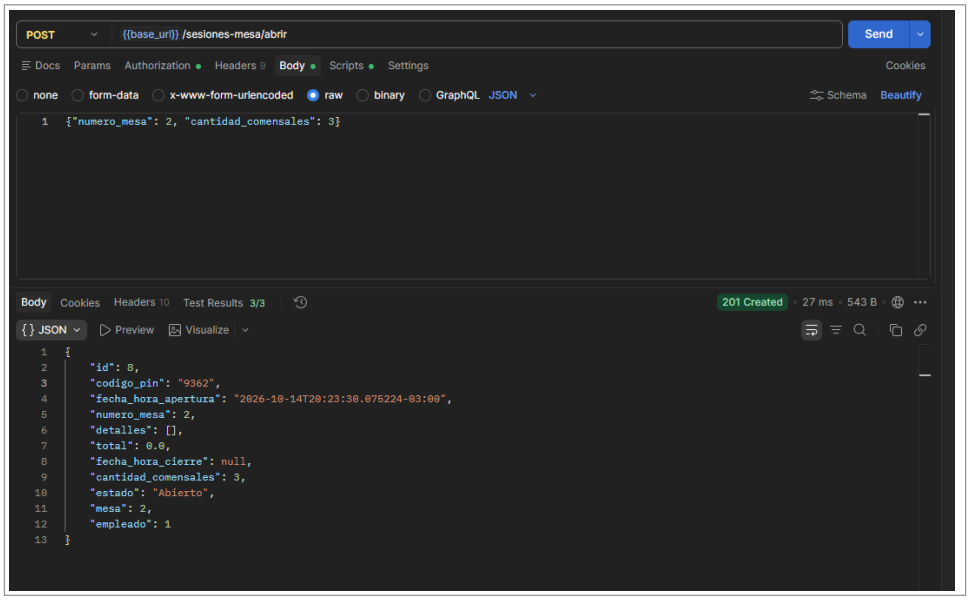
Captura 3. Primera ejecución: 201 Created (sesión id 7).



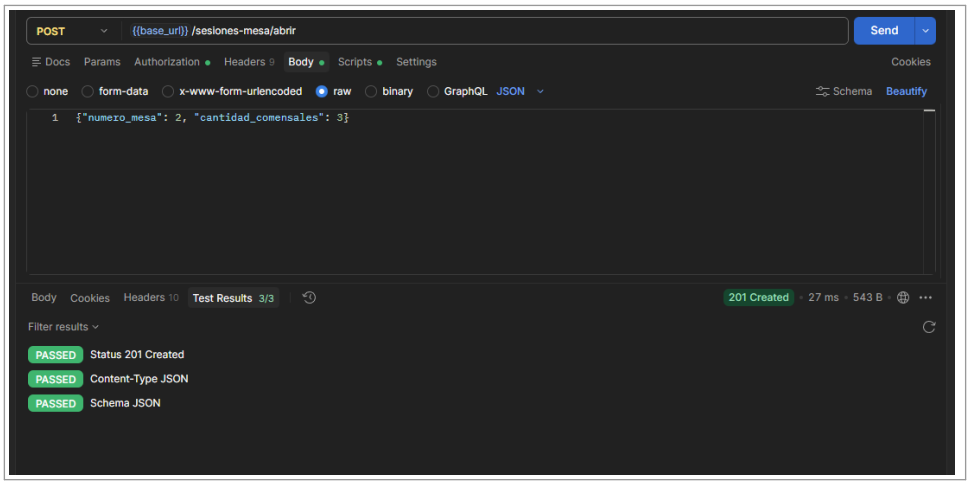
Captura 4. Primera ejecución: Content-Type JSON en FAILED por el script; Status 201 y Schema JSON en PASSED.

Segunda ejecución

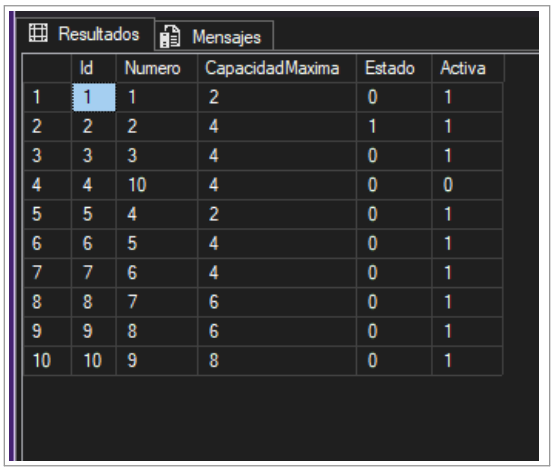
Con el script corregido y la sesión anterior cerrada, se volvió a enviar el request. Resultado: 201 y 3 de 3 verificaciones aprobadas. En la base de datos, la mesa 2 quedó con Estado = 1 (ocupada).



Captura 5. Segunda ejecución: 201 Created, sesión id 8, estado "Abierto".



Captura 6. Test Results: Status 201, Content-Type JSON y Schema JSON en PASSED.


Resultados | Mensajes

	Id	Numero	CapacidadMaxima	Estado	Activa
1	1	1	2	0	1
2	2	2	4	1	1
3	3	3	4	0	1
4	4	10	4	0	0
5	5	4	2	0	1
6	6	5	4	0	1
7	7	6	4	0	1
8	8	7	6	0	1
9	9	8	6	0	1
10	10	9	8	0	1

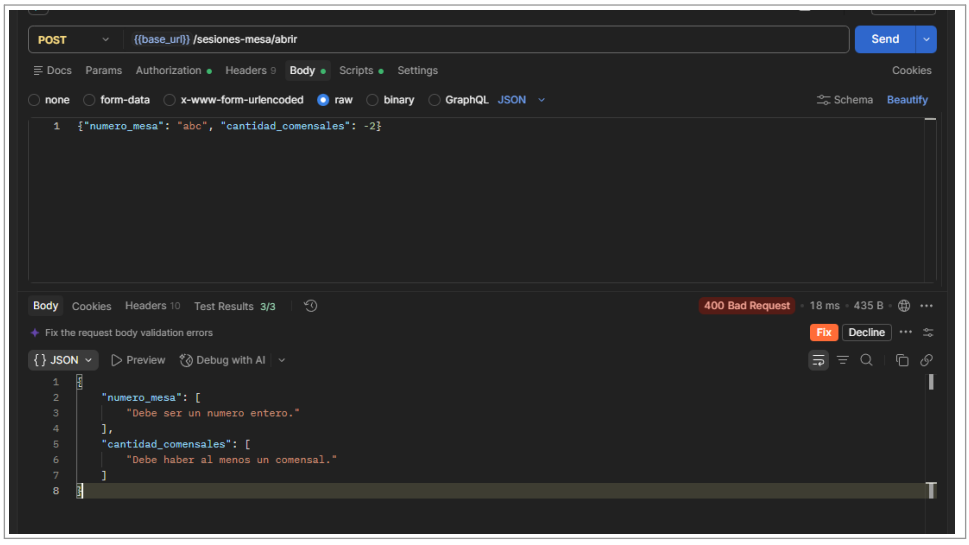
*Captura 7. SQL Server (SELECT * FROM Mesa): la mesa 2 tiene Estado = 1.*

6. Prueba 3: borde 400 (datos corruptos)

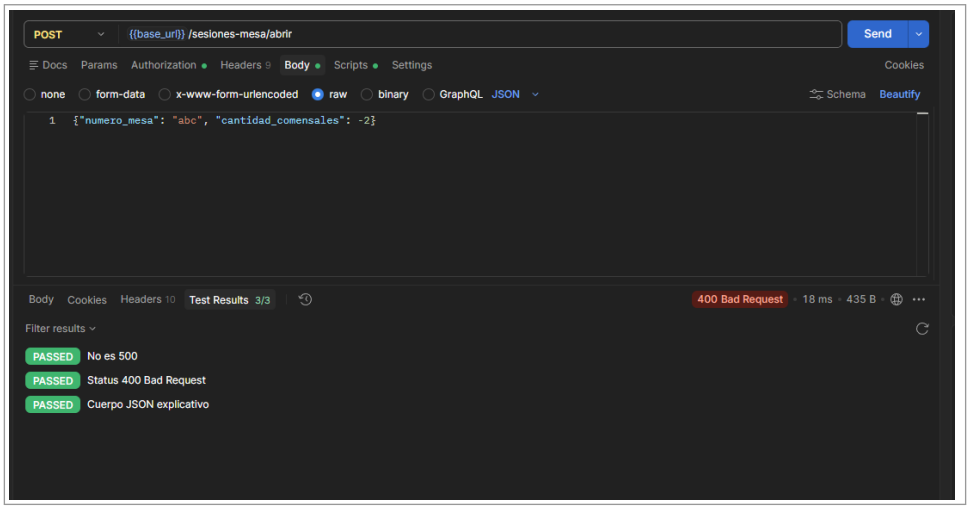
Se enviaron tres entradas corruptas. En los tres casos se esperaba status 400 con un JSON explicativo y que no hubiera error 500 (un 500 indicaría que el servidor falló por un dato del cliente). Cada request tiene 3 verificaciones: no es 500, status 400 y cuerpo JSON.

Request A: datos inválidos

POST /sesiones-mesa/abrir con {"numero_mesa": "abc", "cantidad_comensales": -2}. Obtenido: 400 con un mensaje por campo ("Debe ser un numero entero." y "Debe haber al menos un comensal."). 3 de 3 aprobadas.



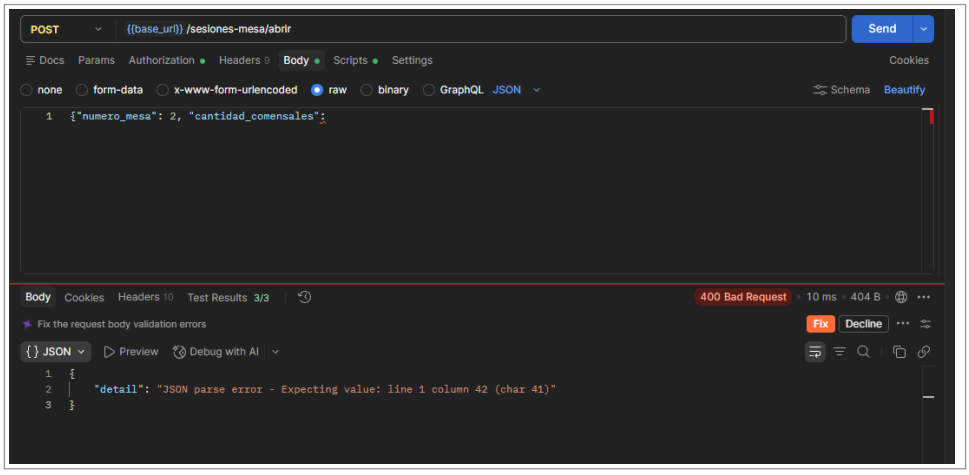
Captura 8. Request A: respuesta 400 con los mensajes de validación.



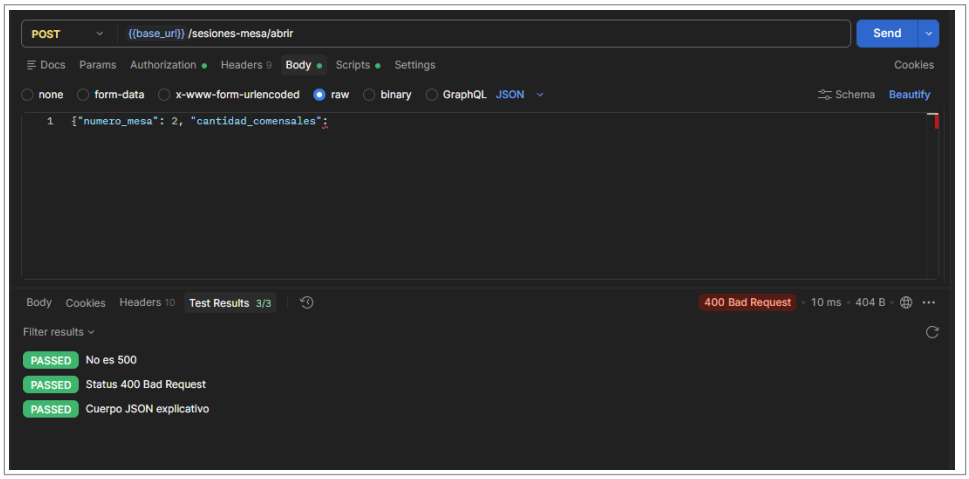
Captura 9. Request A: Test Results, 3 de 3 aprobadas.

Request B: JSON incompleto

POST /sesiones-mesa/abrir con el JSON cortado a propósito: {"numero_mesa": 2, "cantidad_comensales": . Obtenido: 400 con el mensaje "JSON parse error - Expecting value: line 1 column 42". 3 de 3 aprobadas.



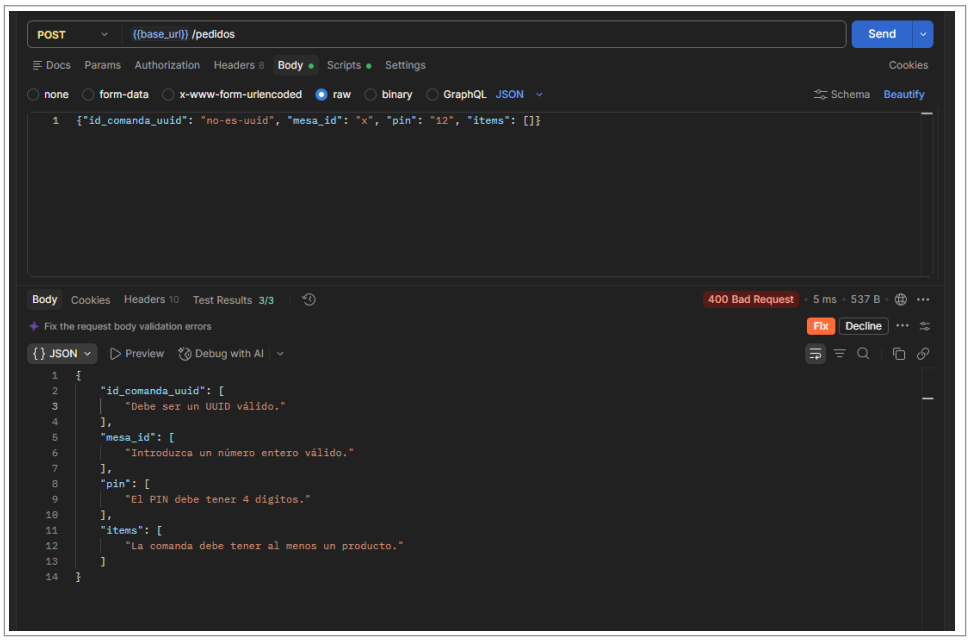
Captura 10. Request B: respuesta 400 por JSON mal formado.



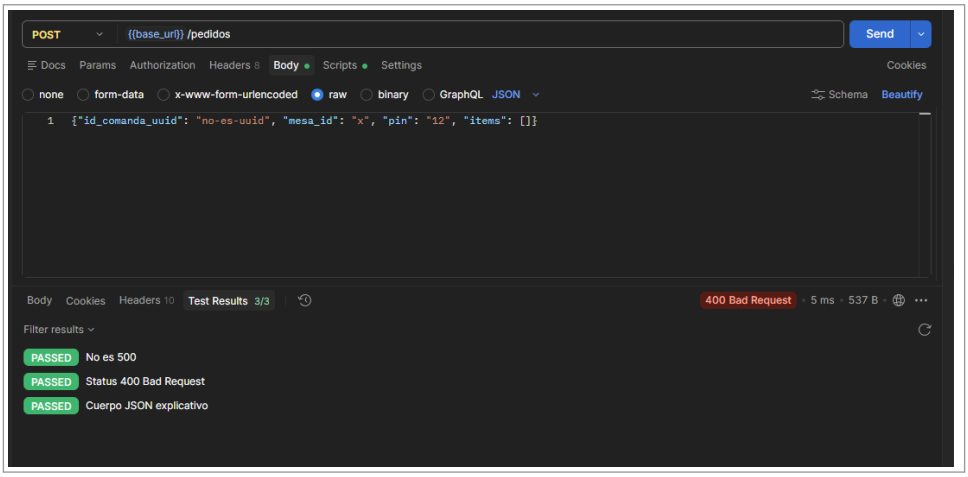
Captura 11. Request B: Test Results, 3 de 3 aprobadas.

Request C: comanda inválida

POST /pedidos (sin autenticación, es un endpoint público) con un UUID inválido, mesa "x", PIN "12" e items vacío. Obtenido: 400 que indica los 4 campos incorrectos (id_comanda_uuid, mesa_id, pin e items). 3 de 3 aprobadas.



Captura 12. Request C: respuesta 400 con un mensaje por cada campo inválido.



Captura 13. Request C: Test Results, 3 de 3 aprobadas.

7. Conclusión

Las tres pruebas pasaron en Postman contra el servidor del proyecto: el 201 devolvió el schema JSON esperado, la regla del subtotal se cumplió y los datos corruptos se rechazaron con 400 sin ningún error 500.

El único resultado erróneo fue un error en el script de una verificación (Content-Type) y no en la API. Quedó documentado en la Prueba 2 y se corrigió.